Classes and Objects in C++

Corso di Programmazione - Prof. Marco Bertini
Università di Firenze

---

1. Perché l'astrazione?

L'**astrazione** è una tecnica fondamentale nella programmazione che permette di:

- Modellare un problema separando i dettagli necessari da quelli non necessari
- Ottenere una separazione tra le operazioni eseguite sui dati e la loro rappresentazione
- Strutturare problemi complessi in entità ben definite tramite i loro dati e operazioni

L'astrazione ci aiuta a gestire la complessità concentrandoci su **cosa** fa un componente, non su **come** lo fa internamente.

---

2. ADT - Abstract Data Type

Un **ADT (Tipo di Dato Astratto)** è una specifica che definisce:

- Un insieme di dati
- Un insieme di operazioni (l'interfaccia dell'ADT) che possono essere eseguite su quei dati

È "astratto" perché è **indipendente dalle implementazioni concrete**. Nel programma, l'ADT è rappresentato da un'interfaccia che protegge l'implementazione sottostante.

Esempio: Stack ADT

- **Dati**: una sequenza $\langle d_1, d_2, \ldots, d_n \rangle$
- **Operazioni**:
  - `push(x)` - inserisce elemento
  - `pop()` - rimuove elemento
  - `top()` - legge elemento in cima
- **Interfaccia**: l'utente vede solo le operazioni, non sa se sotto c'è un array, una lista concatenata, ecc.

Vantaggi: l'implementazione può cambiare senza modificare il codice che usa lo Stack.

---

3. Incapsulamento (Encapsulation)

L'**incapsulamento** è il principio di:

- Nascondere la struttura dati interna
- Fornire accesso solo tramite un'interfaccia ben definita

Questo permette di scegliere o modificare strutture dati appropriate all'ambiente applicativo senza rompere il codice esistente.

Beneficio chiave: separazione tra implementazione e utilizzo.

---

4. Classi e Oggetti

Definizioni fondamentali

Classe: implementazione concreta di un ADT. Definisce:

- **Attributi** (data members): implementano la struttura dati
- **Metodi** (function members): implementano le operazioni

Oggetto: istanza di una classe. Ha:

- Un'identità unica (nome)
- Uno **stato**: rappresentato dai valori degli attributi in un dato momento
- Un **comportamento**: definito dai metodi applicabili

Distinzione importante

- Una **classe** è una rappresentazione astratta di un insieme di oggetti che si comportano identicamente
- Gli **oggetti** (variabili) sono istanziati dalle classi
- Le classi definiscono proprietà e comportamento di insiemi di oggetti

---

5. Programmazione Procedurale vs Object-Oriented

Procedurale (C)	Object-Oriented (C++)
Divisione tra dati e operazioni Focus su strutture dati e funzioni Esempio: struct + funzioni separate	Dati e operazioni combinati negli oggetti Focus su oggetti e loro interazioni Esempio: classe GameCharacter con posizione (dato) e move() (metodo)

Table 1: Confronto tra paradigmi di programmazione

---

6. Open-Closed Principle (OCP)

Definizione (Bertrand Meyer):

> "Le entità software (classi, moduli, funzioni) dovrebbero essere **aperte per l'estensione** ma **chiuse per la modifica**."

Cosa significa?

- **Chiusa per modifica**: se una classe ha un comportamento corretto, il suo codice non deve essere modificato
- **Aperta per estensione**: se serve estendere il comportamento, si può ereditare la classe e sovrascrivere i metodi

Esempio di violazione dell'OCP (codice C)

```
enum ShapeType {circle, square};

struct Shape {
ShapeType itsType;
};

void drawAllShapes(ShapePointer list[], int n) {
for (int i=0; i<n; i++) {
struct Shape* s = list[i];
switch (s->itsType) {
case square:
drawSquare((struct Square*)s);
break;
case circle:
drawCircle((struct Circle*)s);
break;
}
}
}
```

Problema: per aggiungere un triangolo, devo **modificare** `drawAllShapes`, violando l'OCP.

Soluzione OO: usare ereditarietà e polimorfismo (vedremo nelle prossime lezioni).

---

7. Identificazione delle Classi

Approccio pratico

1. Identificare oggetti/entità del mondo reale nel dominio applicativo
2. Identificare gruppi di oggetti che si comportano in modo simile
3. Creare classi come specifiche per quegli oggetti
4. Rimandare decisioni implementative finché non si ha chiaro quali classi servono

Regola empirica

Dai casi d'uso e dalla descrizione del problema, estrai:

- **Sostantivi e frasi nominali** → candidati per classi/oggetti
- **Verbi** → candidati per metodi (azioni)
- **Aggettivi** → candidati per attributi (proprietà)

---

8. Sintassi delle Classi C++

Definizione base

```
class IntStack {
bool push(int value);
bool pop(int* pValue);
void init(int size);
int TOS;
int* buffer;
int size;
}; // Non dimenticare il ; finale!
```

Nota: questa classe è incompleta (mancano livelli di accesso e costruttori).

Estensione delle struct C

Una classe C++ estende il concetto di `struct`:

- Raggruppa variabili (attributi/data members)
- Include funzioni (metodi/function members) che operano nel contesto della classe
- Definisce un nuovo tipo da cui creare istanze (oggetti)

---

9. Il Puntatore `this`

Un oggetto può riferirsi a se stesso usando la keyword **`this`** (puntatore all'oggetto corrente).

Uso implicito

```
IntStack::push(int value) {
if (TOS > size) // equivalente a: if (this->TOS > this->size)
// ...
}
```

Uso esplicito

Utile quando:

- Bisogna passare un riferimento a sé stessi ad altri oggetti
- Si vuole disambiguare tra parametri e attributi con lo stesso nome
- Si implementano pattern come callback o Observer

---

10. Livelli di Accesso

Nelle `struct` C tutti i membri sono pubblici; nelle classi C++ si può controllare la visibilità:

Livello	Visibilità
`public`	Visibile a chiunque abbia un indirizzo/riferimento all'oggetto
`private`	Visibile solo ai metodi della classe (default nelle classi)
`protected`	Visibile ai metodi della classe e delle classi derivate

Table 2: Livelli di accesso in C++

Esempio

```
class IntStack {
public:
bool push(int value);
bool pop(int* pValue);
void init(int size);
```

```
private:
int TOS;
int* buffer;
int size;
};
```

Regole pratiche

- Usare sempre controllo di accesso esplicito
- Non avere data members pubblici
- Usare metodi pubblici getter/setter per accedere ai dati privati
- Molti IDE generano automaticamente getter/setter

---

11. Implementazione dei Metodi

Implementazione esterna (file .cpp)

```
// IntStack.h
class IntStack {
public:
bool push(int value);
// ...
};

// IntStack.cpp
bool IntStack::push(int value) {
// Implementazione del metodo
if (TOS >= size)
return false;
buffer[TOS++] = value;
return true;
}
```

Sintassi: `NomeClasse::nomeMetodo(...) { ... }`

Implementazione inline (nell'header)

Possibile per metodi molto brevi (5-7 righe circa), direttamente nella definizione della classe.

---

12. Attributi e Stato dell'Oggetto

Gli **attributi** (data members):

- Sono visibili dentro i metodi della classe
- Mantengono lo **stato** dell'oggetto
- Rappresentano un contesto condiviso tra i metodi

Vantaggio: semplifica le interfacce rispetto al C (non serve passare puntatori alla struttura in ogni funzione).

Compromesso: aumenta l'accoppiamento (coupling) tra metodi e dati, ma è accettabile se la classe ha **responsabilità coese**.

---

13. Utilizzo di Metodi e Attributi

Sintassi analoga alle `struct`, ma rispettando la visibilità:

```
IntStack s; // Oggetto
IntStack* pS; // Puntatore

pS = &s;

s.push(3); // Notazione punto per oggetti
pS->push(8); // Notazione freccia per puntatori
```

Solo i membri `public` sono accessibili dall'esterno.

---

14. Overloading dei Metodi

È possibile definire più metodi con lo stesso nome ma con **firme diverse** (numero e/o tipo dei parametri).

```
class Printer {
public:
void print(int value);
void print(double value);
void print(const string& text);
};
```

Come funziona

Il compilatore genera simboli diversi per ogni versione (name mangling), distinguendo in base ai tipi dei parametri.

Non è possibile differenziare solo per il tipo di ritorno.

---

15. Overloading degli Operatori

È possibile ridefinire operatori (`+`, `-`, `==`, `<<`, ecc.) per tipi definiti dall'utente.

Esempio: operatore di uguaglianza per Array

```
class Array {
public:
Array(int size);
bool operator==(const Array& right) const;
private:
int size;
int* data;
};

bool Array::operator==(const Array& right) const {
if (size != right.size)
return false;
for (int i=0; i < size; i++) {
if (data[i] != right.data[i])
return false;
}
return true;
}

// Uso
Array A, B;
if (A == B) // equivalente a: A.operator==(B)
// ...
```

Linee guida

- Sovraccaricare operatori solo quando ha senso semantico (es. `==` per confronti, non `*` per stringhe)
- Se sovraccarichi `+`, definisci anche `+=` (non viene generato automaticamente)
- Operatori binari (`+`, `-`) spesso sono funzioni non-membro
- Compound assignment (`+=`, `-=`) sono metodi membro

---

16. Passaggio dei Parametri

In C: pass by value

I valori degli argomenti sono **copiati** nei parametri formali. Per simulare pass by reference si usano puntatori.

In C++: pass by reference

È possibile passare parametri per **riferimento** usando `&`:

```
void add(int a, int b, int& sum) {
sum = a + b; // Modifica direttamente la variabile originale
}

int main() {
int i = 1, j = 2, total = 0;
add(i, j, total);
cout << total << endl; // Output: 3
}
```

Reference vs Puntatori

Una **reference** è essenzialmente un alias/sinonimo:

- Non c'è copia dei dati
- Indicata con `&` dopo il tipo
- Nessun puntatore esplicito, nessuna dereferenziazione necessaria

Passaggio per riferimento costante

```
void processData(const LargeStruct& data) {
// data è passato per riferimento (veloce)
// ma non può essere modificato (const)
}
```

Linee guida per parametri

Situazione	Scelta
Dati grandi, non modificabili	`const T&`
Dati piccoli (int, double)	`T` (per valore)
Parametro da modificare	`T&` (riferimento)
Serve una copia	`T` (per valore)

Table 3: Linee guida per il passaggio parametri

---

17. Variabili Reference

È possibile dichiarare variabili reference, ma devono essere **sempre inizializzate**:

```
int x;
int& y = x; // y è un alias di x
y = 2; // anche x diventa 2

int& z; // ERRORE: reference non inizializzata!

int* p; // Puntatore (può essere non inizializzato)
p = &x;
*p = 3; // x diventa 3
```

Reference vs Puntatori: confronto

Puntatori	Reference
Variabili come le altre	Devono essere inizializzate
Possono essere `void*`	Non esistono reference a void
Possono essere riassegnati	Non possono essere riassegnate
Supportano aritmetica	Non supportano aritmetica
Possono essere `nullptr`	Non possono essere "null"

Table 4: Puntatori vs Reference

Conclusione: le reference sono meno potenti ma **molto più sicure** dei puntatori. Il programmatore non può abusarne facilmente.

Linee guida generali

- Preferire `T&` quando "no argument" non è valido
- Preferire `T*` quando `nullptr` è un valore legittimo
- Mai ritornare reference/puntatori a oggetti locali (dangling reference/pointer!)

---

18. `nullptr` in C++11

Il problema con `NULL` e `0`

In C/C++ classico, `NULL` è semplicemente definito come `0`, causando ambiguità:

```
void test(int*);
void test(int);

test(0); // Quale funzione viene chiamata?
test(NULL); // Ancora ambiguo!
```

La soluzione: `nullptr`

C++11 introduce **`nullptr`**, un tipo fondamentale specifico per puntatori:

```
int* pa = &a;
pa = nullptr;
```

```
if (pa != nullptr) {
// ...
}

test(nullptr); // Chiama test(int*) senza ambiguità
```

Vantaggi:

- Codice più chiaro e sicuro
- Nessuna conversione implicita a `int`
- Risolve ambiguità in overload

Richiede: compilatore C11 (g ≥ 4.2, clang++ ≥ 3.x) con flag appropriati (`-std=c++11`).

---

19. Type Checking Rigoroso

C++ ha un **type checking più rigoroso** rispetto a C:

- In base al cast dei parametri, si determina quale metodo viene eseguito (overloading)
- I puntatori `void*` devono essere esplicitamente convertiti quando assegnati ad altri puntatori (in C compila senza cast)

Questo previene molti errori a tempo di compilazione.

---

20. Creazione degli Oggetti

Creazione statica (stack)

```
Stack myStack; // Costruttore di default
Stack s(100); // Costruttore con parametro
```

Creazione dinamica (heap)

```
Stack* sPtr;
sPtr = new Stack; // Alloca memoria e chiama costruttore
// ...
delete sPtr; // Chiama distruttore e libera memoria
```

Array dinamici

```
int* p = new int[5]; // Array di 5 interi
delete[] p; // Se new ha [], delete deve avere []
```

Nota: `new` calcola automaticamente la dimensione da allocare.

---

21. Costruttori

Un **costruttore** è una funzione membro speciale:

- Ha lo stesso nome della classe
- Non restituisce valori
- Viene invocato automaticamente alla creazione dell'oggetto
- Serve per inizializzare lo stato dell'oggetto

Costruttore di default

Se non definisci costruttori, il compilatore genera un **costruttore di default** senza parametri:

```
class Point {
public:
// Compilatore genera: Point() {}
private:
int x, y;
};

Point p; // OK: chiama costruttore di default
```

Attenzione: `Stack myStack();` dichiara una **funzione**, non un oggetto! Usa `Stack myStack;` invece.

Costruttori con parametri

```
class Stack {
public:
Stack(int s); // Costruttore con parametro
private:
int size;
```

```
int* buffer;
};

Stack s(100); // Chiama Stack(int)
Stack* p = new Stack(50);
```

Regola importante

Se definisci **qualsiasi** costruttore, il compilatore **non genera più** il costruttore di default:

```
class B {
public:
B(int i) { }
};

B b1; // ERRORE: non esiste B()
B b3(123); // OK
```

Linea guida: se definisci costruttori, definisci anche quello di default (se serve).

---

22. Liste di Inizializzazione dei Costruttori

Sintassi **compatta ed efficiente** per inizializzare gli attributi:

```
class Stack {
public:
Stack(int s) : TOS(0), size(s), buffer(new int[s]) {
// Corpo del costruttore (può essere vuoto)
}
private:
int TOS;
int size;
int* buffer;
};
```

Vantaggi

- Equivale a definizione + inizializzazione simultanea
- Più efficiente della semplice assegnazione nel corpo
- **Necessaria** per inizializzare attributi `const` e reference

Importante: usa lo stesso ordine della dichiarazione degli attributi, altrimenti il compilatore genera warning.

Default Initializers (C++11)

Da C++11 puoi inizializzare attributi direttamente nella definizione:

class X {
public:
X() {} // i4, j5
X(int ni) : i(ni) {} // ini, j5

```
int i = 4;
int j {5};  // Sintassi alternativa
```

};

Gli initializers di default vengono usati se il costruttore non li sovrascrive nella lista di inizializzazione.

---

23. Costruttori `explicit`

Il compilatore può fare una **conversione implicita** per risolvere i parametri di una funzione.

Problema

class B {
public:
B(int x);
};

void doSomething(B objB);

doSomething(28); // Il compilatore converte 28 in B(28) implicitamente!

Questo può causare conversioni indesiderate e bug difficili da trovare.

Soluzione: `explicit`

```
class B {
public:
explicit B(int x = 0);
};

doSomething(28); // ERRORE: conversione implicita non permessa
doSomething(B(28)); // OK: conversione esplicita
```

Linea guida: usa `explicit` per costruttori con un solo parametro (o con parametri di default) per evitare conversioni implicite indesiderate. Esiste anche una linea guida Google C++ al riguardo.

---

24. Costruttori Deleganti (C++11)

Un **costruttore delegante** chiama un altro costruttore della stessa classe:

```
class Sales_data {
public:
// Costruttore principale (non delegante)
Sales_data(std::string s, unsigned cnt, double price)
: bookNo(s), units_sold(cnt), revenue(cnt*price) {}

    // Costruttori deleganti
    Sales_data() : Sales_data("", 0, 0) {}
    Sales_data(std::string s) : Sales_data(s, 0, 0) {}

};
```

Vantaggi

- Riduce duplicazione del codice di inizializzazione
- Più sicuro che mettere codice comune in un metodo privato
- Può usare liste di inizializzazione comuni per `const` e reference

Nota: Java ha questa feature da anni usando `this(...)`.

---

25. Distruttori

Un **distruttore** è una funzione membro speciale:

- Nome: `~NomeClasse()`
- Non prende parametri, non restituisce valori
- **Non può essere sovraccaricato** (ne esiste uno solo)
- Chiamato automaticamente quando un oggetto viene distrutto
- Serve per **rilasciare risorse** (memoria, file, connessioni, ecc.)

Esempio

```
class IntStack {
public:
IntStack(int s);
~IntStack();
private:
int* buffer;
};

// Costruttore alloca memoria
IntStack::IntStack(int s) {
buffer = new int[s];
}

// Distruttore rilascia memoria
IntStack::~IntStack() {
delete[] buffer;
}
```

Fondamentale: ogni `new` nel costruttore deve avere il corrispondente `delete` nel distruttore per evitare memory leak.

---

26. Inversione di Responsabilità (Callback)

L'uso di `this` è essenziale quando un oggetto deve passare un riferimento a sé stesso ad altri oggetti.

Pattern Callback

Oggetto A dà un riferimento a sé stesso a **Oggetto B**, che lo userà per invocare metodi su **A**.

Esempio: Observer Pattern

```
class Observer {
public:
void update(Subject* pSubj);
};

class Subject {
public:
Subject(Observer* pObs);
void setState(int aState);
private:
int state;
Observer* pObs;
};

void Observer::update(Subject* pSubj) {
this->state = pSubj->getState();
}

void Subject::setState(int aState) {
this->state = aState;
this->pObs->update(this); // Passa riferimento a sé stesso!
}
```

Schema: A non chiama B per eseguire un'operazione, ma lascia che B chiami A (inversione di controllo).

---

27. Membri Statici

Un **membro statico** è associato alla classe, non alle singole istanze:

- Esiste una sola copia condivisa tra tutti gli oggetti
- Può essere usato senza creare oggetti

Static Data Members

```
class Point {
public:
Point() { x = y = 0; n++; }
~Point() { n--; }
static int count() { return n; }

private:
int x, y;
static int n; // Dichiarazione
};

// Definizione (deve essere fuori dalla classe)
int Point::n = 0;

int main() {
Point a, b;
cout << a.count() << endl; // Output: 2
}
```

Static Function Members

I **metodi statici**:

- Possono essere chiamati senza un oggetto: `Point::distance(a, b)`
- Non hanno accesso a `this`
- Possono accedere solo a membri statici (a meno di ricevere oggetti come parametri)

```
class Point {
public:
static float distance(const Point& a, const Point& b);
};
```

```
// Uso
Point::distance(p1, p2); // Chiamata senza oggetto
```

Inline Static (C++17)

Da C++17 si possono inizializzare membri statici inline:

```
struct MyType {
static inline int value = 10;
static inline std::string name = "Hello";
};
```

Non serve più definizione separata in un file `.cpp`.

---

28. `friend`

La keyword **`friend`** permette a funzioni o classi esterne di accedere ai membri privati di una classe.

Esempio

```
class Point {
private:
int x, y;
public:
friend bool operator==(Point a, Point b);
};

bool operator==(Point a, Point b) {
return (a.x == b.x && a.y == b.y); // Accesso a membri privati
}
```

Quando usarla

- In situazioni molto speciali, ad esempio per overload di operatori I/O (`operator<<`, `operator>>`)
- Quando non è possibile o desiderabile fornire accessor pubblici

Attenzione: `friend` riduce l'incapsulamento. Usarla solo quando **realmente necessaria** e se non esistono alternative migliori.

Alternativa senza friend

Spesso si può evitare friend trasformando la funzione in metodo membro:

```
class Point {
public:
bool operator==(Point right) {
return (this->x == right.x && this->y == right.y);
}
};
```

---

29. Classi Annidate (Inner/Nested Classes)

Una **inner class** (classe annidata) è dichiarata completamente dentro un'altra classe.

Scopo

- Incapsulare dettagli implementativi
- Nascondere strutture interne agli utenti della classe esterna

Esempio: Lista con nodi interni

```
class List {
public:
List() : head(nullptr), tail(nullptr) {}

private:
class Node { // Classe interna, non visibile all'esterno
public:
int data;
Node* next;
Node* prev;
};

    Node* head;
    Node* tail;

};
```

Gli utenti di `List` non vedono `Node`, che rimane un dettaglio implementativo privato.

Accesso ai membri (C++11)

Da C++11, le classi annidate sono trattate come **friend** dell'esterna, quindi possono accedere ai suoi membri privati (usando puntatori/riferimenti espliciti).

---

30. Single Responsibility Principle (SRP)

Definizione:

> "Ogni classe dovrebbe avere una sola responsabilità e quindi un solo motivo per cambiare."

Una **responsabilità** è qualcosa che una classe "sa" o "fa".

Esempio di violazione

```
class Report {
public:
void generateContent(); // Responsabilità: logica del report
void printReport(); // Responsabilità: formattazione output
};
```

Problema: due motivi per modificare la classe:

1. Cambia la logica del contenuto → modifica `generateContent()`
2. Cambia il formato di stampa → modifica `printReport()`

Soluzione

Separare le responsabilità in classi diverse:

```
class Report {
public:
void generateContent();
};

class ReportPrinter {
public:
```

```
void print(const Report& report);
};
```

Test pratico per SRP

Per ogni metodo, prova a dire: "La classe X fa Y".

Se alcune frasi non hanno senso o sembrano fuori contesto, quei metodi probabilmente appartengono ad altre classi.

Esempio Automobile:

- `Automobile::accelerate()` ✓ OK
- `Automobile::brake()` ✓ OK
- `Automobile::changeTires()` ✗ responsabilità del meccanico
- `Automobile::wash()` ✗ responsabilità dell'autolavaggio

Conclusioni

Questo documento ha coperto i concetti fondamentali di **classi e oggetti in C++**:

- Astrazione e ADT come basi della programmazione OO
- Incapsulamento e information hiding
- Sintassi e semantica delle classi C++
- Costruttori, distruttori e gestione del ciclo di vita degli oggetti
- Membri statici e loro utilizzo
- Principi di design: Open-Closed Principle e Single Responsibility Principle
- Pattern avanzati: callback, friend, classi annidate

Questi concetti costituiscono le fondamenta della programmazione object-oriented in C++ e preparano per argomenti più avanzati come ereditarietà, polimorfismo e template.

Riferimenti:

- Materiale del corso di Programmazione, Prof. Marco Bertini, Università di Firenze
- Bertrand Meyer, "Object-Oriented Software Construction"

- Google C++ Style Guide